

Spring WebFlux -raportti

Kuinka Spring WebFlux toimii?

Spring WebFlux on Spring Framework 5:n mukana tullut reaktiivinen web-framework. Se perustuu Reactor-kirjastoon ja käyttää ei-blokkaavaa I/O-mallia.

Perinteisessä Spring MVC:ssä jokainen HTTP-pyyntö käyttää yleensä omaa säiettä. WebFlux toimii eri tavalla: se käyttää tapahtumapohjaista mallia (event loop), jossa pieni määrä säikeitä pystyy käsittelemään suuren määrän pyyntöjä samanaikaisesti. Tämän vuoksi WebFlux soveltuu hyvin sovelluksiin, joissa on paljon samanaikaisia käyttäjiä tai asynkronisia operaatioita.

WebFlux käyttää oletuksena Netty-palvelinta Tomcatin sijaan.

WebFluxin ohjelmointimallit

WebFlux tarjoaa kaksi erilaista tapaa rakentaa sovellus.

Annotaatiopohjainen malli

Tämä toimii lähes samalla tavalla kuin Spring MVC. Käytössä ovat tutut annotaatiot kuten `@RestController` ja `@GetMapping`, joten Spring MVC:tä käyttäneille tapa on helposti omaksuttava.

Funktionaalinen malli

Tässä mallissa käytetään `RouterFunction`- ja `HandlerFunction`-rakennetta. Reititys ja pyyntöjen käsittely erotetaan omiin luokkiinsa, mikä tekee rakenteesta hieman funktionaalisemman.

Tiedostojen roolit

SpringWebFluxApplication.java

Sovelluksen käynnistyspiste. `@SpringBootApplication` käynnistää Spring Bootin automaattiset konfiguraatiot.

Book.java

MongoDB-dokumenttiluokka. `@Document`-annotaatio määrittelee luokan MongoDB-kokoelmaksi ja `@Id` määrittelee dokumentin pääavaimen.

BookRepository.java

Reaktiivinen repository, joka laajentaa `ReactiveMongoRepository`-rajapintaa. CRUD-operaatiot palauttavat `Mono`- ja `Flux`-tyyppejä.

BookController.java

Annotaatiopohjainen REST-controller. Käyttää tuttuja Spring-annotaatioita, mutta palauttaa reaktiivisia tietotyyppejä.

BookHandler.java

Funktionaalisen mallin handler-luokka, joka käsittelee HTTP-pyynnöt ja palauttaa ServerResponse-olioita.

BookRouter.java

Määrittelee URL-reititykset funktionaalisessa mallissa. Reitit yhdistetään oikeisiin handler-metodeihin.

application.properties

Konfiguraatiotiedosto, jossa määritellään esimerkiksi embedded MongoDB:n asetukset.

Uudet annotaatiot ja reaktiiviset tyypit

@Document

Spring Data MongoDB:n annotaatio, joka vastaa käytännössä JPA:n @Entity-annotaatiota.

@Id

Määrittelee dokumentin pääavaimen. MongoDB voi luoda ObjectId-arvon automaattisesti.

Flux<T> ja Mono<T>

Nämä eivät ole annotaatioita vaan reaktiivisia tietotyyppejä.

- Flux<T> kuvaa useita arvoja (0...N)
- Mono<T> kuvaa yhtä arvoa tai tyhjää tulosta (0...1)

Molemmat toimivat asynkronisesti ja mahdollistavat ei-blokkaavan tietojenkäsittelyn.

Miten WebFlux eroaa perinteisestä Spring MVC:stä?

Palautustyytit

Perinteisessä Spring MVC:ssä metodit palauttavat yleensä tavallisia olioita, kuten List<Book> tai Book. WebFluxissa palautetaan Mono- tai Flux-olioita.

Palvelinteknologia

Spring MVC käyttää yleensä Tomcatia ja blokkaavaa I/O:ta. WebFlux käyttää oletuksena Nettyä ja ei-blokkaavaa I/O:ta.

Tietokantaoperaatiot

WebFlux hyödyntää reaktiivista MongoDB-ajuria, joka ei varaa säiettä odottamaan tietokantaoperaation valmistumista. Perinteinen JPA toimii blokkaavasti.

Skaalautuvuus

WebFlux pystyy käsittelemään suuren määrän samanaikaisia pyyntöjä pienemmällä määrällä säikeitä, mikä tekee siitä tehokkaan kuormitetuissa järjestelmissä.

Ohjelmoinnin monimutkaisuus

Reaktiivinen ohjelmointi on usein vaikeammin hahmotettavaa kuin perinteinen Spring MVC. Sen käyttö vaatii hieman erilaisen ajattelutavan ja asynkronisen ohjelmoinnin ymmärtämistä.

Funktionaalinen ohjelmointimalli

WebFlux tarjoaa myös funktionaalisen Router + Handler -mallin, jota ei perinteisessä Spring MVC:ssä yleensä käytetä.